

UCS503 – Software Engineering Project
Academic Year 2026–2027



MedFinder: Real-Time Medicine Availability & Store Locator

Dynamic Inventory Search and Store-Level
Medicine Reservation Platform

Submitted by:

1024030436 Madhav Gupta

1024030435 Sourabh Chhabra

1024030446 Arpit Yadav

B.E. Third Year, Computer Science & Engineering

Submitted to:

Dr. Jeelani Asif

Department of Computer Science and Engineering

Mid-Semester Evaluation Report

Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala

Contents

1	Introduction	2
1.1	Background and Context	2
1.1.1	Problem Statement	2
1.2	Project Objectives	2
2	Methodology and Design	3
2.1	System Architecture	3
2.1.1	High-Level Design	3
2.1.2	Technical Stack	3
2.2	Implementation Details	3
2.2.1	Module 1: Core Functionality	3
2.2.2	Module 2: Secondary Features	4
3	Results and Evaluation	5
3.1	Testing Methodology	5
3.2	Performance Metrics	5
3.3	Analysis	5
4	Conclusion	6
4.1	Summary of Work	6
4.2	Key Findings	6
4.3	Future Work and Recommendations	6
4.4	Final Remarks	6

Chapter 1

Introduction

1.1 Background and Context

Access to the right medicine at the right time is a basic requirement of healthcare, yet no medical store stocks every medicine at all times. Patients routinely discover – only after travelling to a pharmacy – that the medicine they need is out of stock, unavailable in the required strength, or simply not carried by that store. This is true both for specialised or prescription-only drugs and, at times, for common over-the-counter medicines that have temporarily run out.

This problem becomes especially serious during urgent or emergency situations, and disproportionately affects elderly patients, chronically ill patients, and caregivers who cannot easily visit several pharmacies in sequence. At present there is no simple, centralised way for a patient to check, before leaving home, which nearby recognised pharmacy actually has a given medicine in stock. MedFinder is proposed as a web application that addresses this gap directly: it lets a user search for a medicine and instantly see which verified, nearby pharmacies currently stock it, along with price and distance, so that the user can go straight to a store that can actually fulfil their need.

1.1.1 Problem Statement

Patients lack a centralised, real-time way to know which nearby pharmacy has a required medicine in stock. This leads to wasted time, unnecessary travel, and, in urgent cases, delayed treatment. Even after a patient identifies a store that reportedly has the medicine, there is no guarantee it will still be available by the time they arrive, since stock can be sold out in the interim. This uncertainty affects both rare, specific medicines and everyday medicines alike, and stems fundamentally from the lack of visibility into store-level inventory that patients currently have. MedFinder resolves this gap through a centralised inventory-search system that queries live, store-level stock records and returns a ranked list of verified pharmacies that actually hold the requested medicine.

1.2 Project Objectives

The core objectives of the MedFinder project comprise:

- **Centralised Inventory Listing:** Provide a single platform listing recognised medical stores in a given area along with their live medicine inventory.
- **Search, Filter & Rank:** Allow users to search for a specific medicine and view results filtered and sorted by nearby distance and lowest price.
- **Travel-Time Reduction:** Reduce the time and travel patients currently spend visiting or calling multiple pharmacies to locate a medicine.
- **Reservation Mechanism:** Offer a Reserve feature so a user can hold a medicine at a chosen store until they arrive, reducing the risk of it selling out in the meantime.
- **Trust & Verification:** Ensure trust and safety by only listing pharmacies that have gone through a verification/registration process.
- **Scalable Engineering Stack:** Build the system on a reliable, maintainable engineering stack that can scale from a small prototype to a citywide deployment.

Chapter 2

Methodology and Design

2.1 System Architecture

2.1.1 High-Level Design

MedFinder follows a standard three-tier web architecture organized into the **Client Layer**, the **Application Layer**, and the **Data Layer**. Figure 2.1 illustrates the high-level architecture.

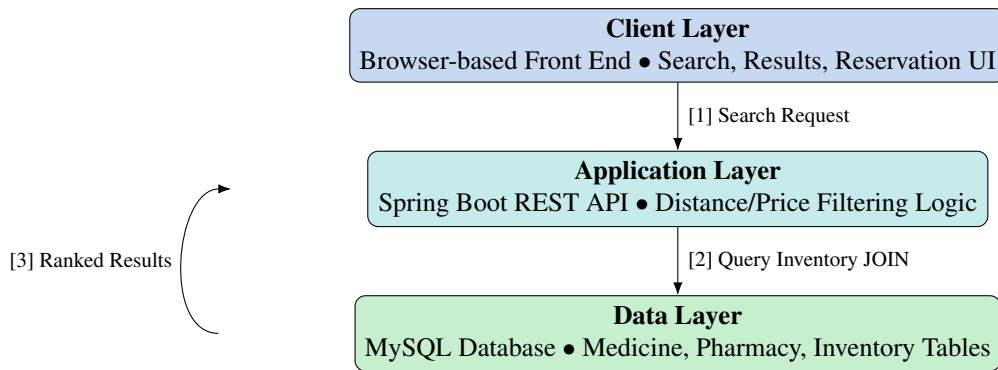


Figure 2.1: MedFinder High-Level Three-Tier Architecture

The core workflow is: a user submits a medicine name from the client → the backend queries the Inventory table joined with Medicine and Pharmacy → matching records are ranked by distance (using the user’s coordinates) and price → the ranked list is returned to the client for display, with the option to reserve an item at a chosen pharmacy.

2.1.2 Technical Stack

- **Backend:** Java 25 with Spring Boot (Spring Web, Spring Data JPA) for the REST API layer.
- **Persistence:** Hibernate ORM over a MySQL 8 database, managed locally through phpMyAdmin.
- **Connection Pooling:** HikariCP, bundled with Spring Boot, for efficient database connections.
- **Build Tool:** Apache Maven (via the IntelliJ-bundled Maven wrapper, `mvnw.cmd`).
- **Frontend Tooling:** HTML, CSS, Javascript, Node.js for building and serving the web client that consumes the REST API.
- **Development Environment:** IntelliJ IDEA for the Spring Boot backend and VS Code for auxiliary scripting and frontend work, on Windows.
- **Version Control:** Git, for tracking changes as the prototype evolves.

2.2 Implementation Details

2.2.1 Module 1: Core Functionality

The core module implements the data model and search capability that the rest of the application depends on. Figure 2.2 shows the request-processing pipeline for a search query.

- **Medicine entity/repository** – stores medicine name and description (e.g., Paracetamol, Ibuprofen).
- **Pharmacy entity/repository** – stores pharmacy name, address/area, and latitude/longitude used for distance calculations.
- **Inventory entity/repository** – links a Medicine to a Pharmacy with price, stock quantity, and a last-updated indicator, and is the table actually queried for availability.
- **REST endpoints** – `GET /api/medicine` and `GET /api/pharmacy` expose the base catalogues, while `GET /api/search?medicine={name}` returns matching inventory records for a given medicine name.

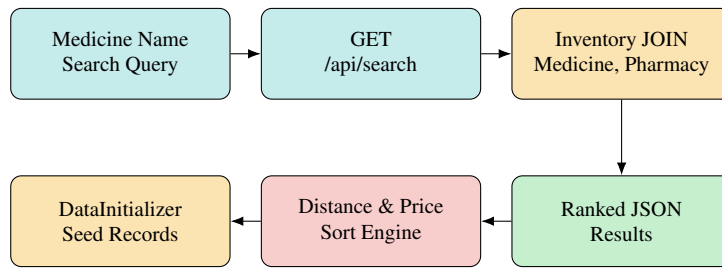


Figure 2.2: Core Search and Inventory Query Pipeline

- **DataInitializer** – seeds the database with sample medicines, pharmacies, and inventory records on first run, which was used to validate the schema and endpoints end-to-end.

2.2.2 Module 2: Secondary Features

Building on the core module, the following features extend MedFinder toward its full proposed scope. Figure 2.3 shows how a search result flows into filtering and reservation.



Figure 2.3: Secondary Feature Flow: Filtering, Reservation, and Verification

- **Distance and price filtering/sorting** of search results, so users can prioritise the nearest or cheapest available option.
- **Reserve feature** allowing a user to place a temporary hold on an in-stock item at a chosen pharmacy until they arrive.
- **Verified pharmacy registration**, so only recognised, authentic stores can be listed and appear in search results.
- **Planned refinements** such as fuzzy text matching on medicine names (to tolerate spelling variants) and periodic inventory refresh to keep stock counts current.

Chapter 3

Results and Evaluation

3.1 Testing Methodology

The backend was tested incrementally, endpoint by endpoint, directly through the browser and by inspecting the underlying MySQL tables in phpMyAdmin. Each endpoint (`/api/medicine`, `/api/pharmacy`, `/api/search`) was checked in isolation to confirm it returned the expected JSON before moving on to the next, which made it straightforward to localise issues – for example, an empty result from `/api/search` was traced back to missing rows in the pharmacy and inventory tables rather than a code defect.

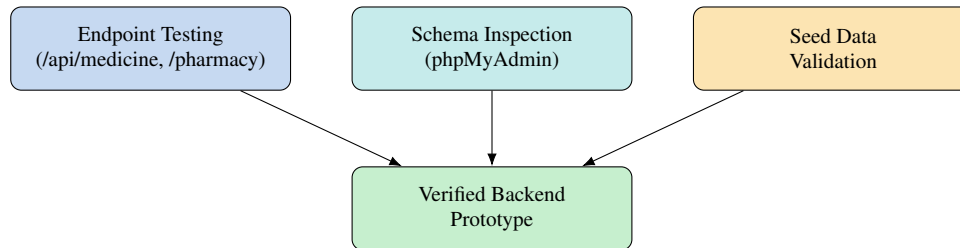


Figure 3.1: Incremental Endpoint Testing and Schema Validation Framework

This process also surfaced and resolved a schema-level issue during development: an existing medicine table had been created without `AUTO_INCREMENT` on its `id` column, which caused insert failures once Hibernate attempted to generate IDs automatically. The fix was to drop the affected prototype tables and allow Hibernate to recreate them with the correct schema, after which seeding and querying worked as expected.

3.2 Performance Metrics

Formal performance testing (load testing, response-time benchmarking, and search-accuracy measurement against a larger dataset) has not yet been carried out on the prototype and is planned as a next step once the frontend is complete and a larger sample dataset is loaded. Table 3.1 summarises the metrics intended to be captured, along with their current status.

Evaluation Metric	Target Specification	Status
Average API response time for <code>/api/search</code> (typical & peak load)	≤ 500 ms	Pending
Accuracy of medicine-name matching (incl. spelling variants)	$\geq 90.0\%$	Pending
Correctness of distance-based ranking vs. known coordinates	100%	Pending
System behaviour as pharmacy/inventory records scale up	Stable throughput	Pending

Table 3.1: MedFinder Planned Performance Evaluation Metrics

3.3 Analysis

At the current stage, the backend prototype reliably exposes medicine, pharmacy, and inventory data through REST endpoints, and the search endpoint correctly returns matching inventory once the database is seeded correctly. This confirms that the core data model and query logic are sound. The main remaining work is integrating a complete frontend against these endpoints, implementing the Reserve and verification features fully, and then subjecting the combined system to the performance and accuracy testing described above.

Chapter 4

Conclusion

4.1 Summary of Work

This project set out to build MedFinder, a web application that lets patients search for a medicine and instantly see which nearby, verified pharmacies have it in stock, along with price and distance. A three-tier architecture was designed with a Spring Boot REST API and a MySQL database, and a working backend prototype was implemented covering the Medicine, Pharmacy, and Inventory data model, seed data, and the core search endpoint, with the schema and endpoints validated through direct testing.

4.2 Key Findings

- A relational schema of Medicine, Pharmacy, and Inventory tables is sufficient to model real-time, store-level availability and supports the search-by-medicine query pattern MedFinder needs.
- Spring Boot with Spring Data JPA and Hibernate allows the core API to be built and iterated on quickly, though schema-generation details (such as `AUTO_INCREMENT` columns) need care when the database already contains tables from earlier iterations.
- Testing endpoints incrementally, and inspecting the database directly, was an effective way to isolate issues between the application layer and the data layer.

4.3 Future Work and Recommendations

For the final project phase, the team will implement:

- Complete the frontend so users can search, filter by distance/price, and reserve medicine through a full interface rather than raw API calls.
- Implement the pharmacy verification/registration workflow so only recognised stores can list inventory.
- Add fuzzy/approximate text matching on medicine names to make search more forgiving of typos and brand-name variants [?].
- Introduce a mechanism for pharmacies to keep inventory data current (manual updates initially, with API-based or automated sync as a longer-term goal).
- Carry out the performance and accuracy evaluation outlined in Section 3.2 once the system is feature-complete.
- Consider CI/CD automation and basic scalability planning (e.g., indexing, caching) before any pilot deployment beyond a single city [?].

4.4 Final Remarks

MedFinder addresses a genuine and common problem – not knowing, in advance, which nearby pharmacy actually has a needed medicine in stock. The prototype work completed so far demonstrates that the core idea is technically workable: a searchable, store-level inventory system can be built on a conventional and well-supported stack. The remaining work is primarily one of completing the user-facing features and validating the system’s accuracy and performance at a realistic scale.